

Lección 11: Playwright Visibility

Visibilidad con Herramientas Existentes

Agenda

- Visibilidad Cero vs Rica
 - Filosofía: No Custom Tools
 - Configuración de Observabilidad
 - Herramientas Built-in
 - Casos de Uso
-

El Problema

✗ Sin Visibilidad (Black Box - caja negra):

1. Ejecutar: "npx playwright test"
2. Resultado: "✗ 3 tests fallaron"
3. Developer: "¿Qué? ¿Dónde? ¿Por qué?"
4. Debugging: console.log + adivinando
5. Tiempo perdido: 2-3 horas por falla

📊 Métricas: CERO
🔍 Visibilidad: NINGUNA
🐛 Debugging: DOLOROSO

✓ Con Visibilidad Rica

Observable Testing (testing observable):

1. Ejecutar: "npx playwright test"
2. Resultado: "❌ 3 fallaron" + Reportes ricos
3. Developer: Click en reporte → Ver punto de falla
4. Debugging: Traces + screenshots + videos
5. Fix: Enfocado con contexto exacto
6. Tiempo: 15-30 minutos



Métricas: Tasas de éxito, tiempos



Visibilidad: Detalles completos de ejecución



Debugging: Visual + contextual

Filosofía: Usar Herramientas Integradas

❌ Custom Tools (herramientas personalizadas):

- Construir dashboard personalizado
- Crear scripts de métricas
- Mantener infraestructura personalizada
- Pérdida de tiempo

✅ Built-in Tools (herramientas integradas):

- Playwright HTML reports (existente)
- Trace viewer (existente)
- JSON reporters (existente)
- Video/screenshots (existente)

Por qué: Mantenidas por expertos, documentadas, actualizadas automáticamente

Configuración Rica

```
// playwright.config.ts
export default defineConfig({
  // 📊 REPORTING: Múltiples formatos
  reporter: [
    ['html', { outputFolder: 'playwright-report' }],
    ['json', { outputFile: 'test-results/results.json' }],
    ['line'],
  ],
});
```

```
],  
// 🐛 RICH DEBUGGING (depuración rica)  
use: {  
  screenshot: 'only-on-failure',  
  video: 'retain-on-failure',  
  trace: 'retain-on-failure',  
},  
  
// 🌐 MULTI-BROWSER (multi-navegador)  
projects: [{ name: 'chromium' }, { name: 'firefox' }, { name: 'mobile-chr  
}]
```

Métricas Automáticas

1. Success Rate (tasa de éxito):

 Total tests: 45
 Pasaron: 42 (93%)
 Fallaron: 3 (7%)
 Duración promedio: 2.3s
 Total: 1m 45s

2. Cross-Browser (multi-navegador):

 Chromium: 15/15 (100%) 
 Firefox: 14/15 (93%) 
 Mobile: 13/15 (87%) 

3. Performance (rendimiento):

✂ Login: 1.2s (rápido)
✂ Checkout: 4.8s (medio)
✂ Cart: 12.3s (lento - ¡investigar!)

Tool 1: HTML Report

npx playwright test --reporter=html

Características automáticas:

-  Overview (vista general): Desglose pass/fail
-  Execution timeline (línea de tiempo de ejecución)
-  Failure screenshots (capturas de pantalla de fallas)
-  Video recordings (grabaciones de video)
-  Performance metrics (métricas de rendimiento)
-  Cross-browser matrix (matriz multi-navegador)

Todo integrado, cero configuración

Tool 2: Trace Viewer

```
npx playwright show-trace test-results/trace.zip
```

Features (características):

-  Step-by-step timeline (línea de tiempo paso a paso)
-  Screenshot every action (captura en cada acción)
-  Network requests/responses (peticiones/respuestas de red)
-  DOM snapshots (instantáneas del DOM)
-  Console logs (logs de consola)
-  Hover states captured (estados hover capturados)

Ejemplo:

Timeline:

```
00:00.0 → Navegar a localhost
00:01.2 → Click "Add to Cart"
00:01.8 → Esperar carrito (FALLÓ AQUÍ)
00:02.5 → API retornó error 500
00:02.5 → Console: "TypeError: Cannot read 'price'"
```

 Causa Raíz: Bug en API backend

Tool 3: JSON Metrics

```
{
  "stats": {
```

```
    "total": 45,  
    "expected": 42,  
    "unexpected": 3,  
    "duration": 105423  
  },  
  
  "tests": [  
    {  
      "title": "should complete purchase",  
      "status": "passed",  
      "duration": 2341  
    }  
  ]  
}
```

Acceso programático a todas las métricas

Caso 1: Bug Hunt (caza de bugs)

Sin visibilidad:

❌ "checkout" falló
Proceso: Re-ejecutar 10x, agregar logs, adivinar
Tiempo: 3 horas

Con visibilidad:

❌ "checkout" falló

1. Revisar HTML report → captura de falla
2. Abrir trace → momento exacto
3. Network tab → Payment API error 402
4. Timeline → Botón clickeado antes que API esté lista
5. Causa raíz: Race condition (condición de carrera)

Fix: Agregar wait apropiado
Tiempo: 20 minutos

Caso 2: Performance Regression (regresión de rendimiento)

Métricas automáticas revelaron:

 Checkout Performance:

Semana 1: 2.3s 

Semana 2: 2.8s 

Semana 3: 4.1s  (iregresión!)

Semana 4: 7.2s   (icrítico!)

 Análisis de trace:

- Llamadas API 3x más lentas
- Múltiples peticiones redundantes
- Llamadas innecesarias del frontend

Resultado: Detectado antes de impacto mayor

Caso 3: Cross-Browser Issues (problemas multi-navegador)

Comparación de navegadores:

Test: "Add to cart"

 Chromium:  1.2s

 Firefox:  Falló en "click quantity"

 Mobile:  2.1s

Firefox trace:

- Click disparado correctamente
- Botón encontrado
- Error: "scrollIntoView is not a function"
- Causa raíz: Compatibilidad con Firefox

Fix: Agregar scroll compatible con Firefox

Métricas sin Custom Scripts

Extracción básica de métricas:

```
# Usar JSON existente + herramientas estándar
# --reporter=json: genera reporte en formato JSON
# > results.json: redirecciona output al archivo results.json
npx playwright test --reporter=json > results.json

# Estadísticas
# cat: lee archivo y muestra contenido
# jq: herramienta para procesar/filtrar JSON
# '.stats': extrae solo la sección "stats" del JSON
cat results.json | jq '.stats'

# Datos de rendimiento
# '.tests[].duration': extrae duración de cada test
# sort -n: ordena números de menor a mayor (n = numeric)
cat results.json | jq '.tests[].duration' | sort -n

# Tests fallidos
# select(.status == "failed"): filtra solo tests con status "failed"
cat results.json | jq '.tests[] | select(.status == "failed")'
```

CI/CD Integration

```
# .github/workflows/e2e.yml
name: E2E with Visibility
on: [push]

jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm ci
      - run: npx playwright test --reporter=html,json
      - name: Subir resultados
        if: always()
        uses: actions/upload-artifact@v3
        with:
          name: playwright-results
          path: |
            playwright-report/
            test-results/
```

Anti-Patterns (1/2)

❌ Custom Dashboard (dashboard personalizado):

- Construir HTML personalizado
- Scripts de métricas personalizados
- Mantener infraestructura
- Tiempo vs features

✅ Usar Built-in (integrado):

- HTML report tiene todo
 - Trace viewer para debugging
 - JSON para automatización
 - Cero mantenimiento
-

Anti-Patterns (2/2)

❌ Over-Engineering (sobre-ingeniería):

- Pipelines de analytics complejos
- Sistemas de alertas personalizados
- Agregación de logs personalizada

✅ Simple & Effective (simple y efectivo):

- Screenshots muestran qué
- Traces muestran por qué
- Videos muestran cómo
- JSON provee datos

Ejercicio 1: Analizar HTML Report

Prompt:

Actúa como un ingeniero de QA analizando métricas de performance en E2E tes

CONTEXT0: Observable Testing (testing observable) usa visibilidad rica para debugging rápido. Sin visibilidad = Black Box (caja negra): "❌ 3 tests fal sin contexto, debugging doloroso (2-3 horas). Con visibilidad: HTML reports muestran duración de cada `test`, screenshots, traces. Performance bottleneck (cuellos de botella): tests lentos ralentizan CI. HTML reporter built-in de Playwright genera reports sin código extra. Filosofía: "No Custom Tools", u herramientas existentes.

TAREA: Ejecuta E2E tests, analiza HTML report para identificar `test` más len

COMANDOS:

1. Ejecutar tests con HTML reporter: `npm test:e2e --reporter=html`
2. Abrir report interactivo: `npx playwright show-report`

ANÁLISIS REQUERIDO:

- Revisar duración de cada `test` en report
- Identificar `test` más lento (nombre + duración en ms)
- Calcular % del tiempo total que consume
- Crear desglose de tiempos de todos los tests
- Identificar posible causa del bottleneck

OUTPUT FORMAT:



Análisis de Performance E2E:

Test más lento: [nombre]

Duración: [ms] ([%] del total)

Desglose completo:

Bottleneck identificado:

[Razón por qué es lento – ej: múltiples interacciones DOM]

VALIDACIÓN: Report debe abrir en browser mostrando lista de tests con durac

Aprende: HTML reports built-in revelan bottlenecks sin instrumentación custom

Ejercicio 2: Explorar Trace Viewer

Prompt:

Actúa como un developer debuggeando `test` flaky con Trace Viewer de Playwrig

CONTEXTO: Flaky tests (tests intermitentes) fallan aleatoriamente, difícil debuggear. Debugging tradicional: agregar `console.log` + re-ejecutar esperan falla (tedioso). Trace Viewer de Playwright = time-travel debugging: capturar timeline completo de acciones, screenshots en cada paso, network requests, console logs, DOM snapshots. Configuración trace: `'on'` (siempre), `'retain-on-failure'` (solo si falla), `'on-first-retry'` (al reintentar). Built-in tool = sin instalación extra.

TAREA: Configura traces y explora Trace Viewer para entender debugging visu

CONFIGURACIÓN:

- Archivo: `playwright.config.ts`
- Sección: `use`
- Configurar:
 - `trace: 'on'` (captura siempre para práctica)
 - `screenshot: 'only-on-failure'` (screenshots solo si falla)
 - `video: 'retain-on-failure'` (videos solo si falla)

EJECUCIÓN:

1. Configurar `playwright.config.ts` con `trace: 'on'`
2. Ejecutar tests: `npm test:e2e`
3. Localizar trace file: `test-results/[test-name]/trace.zip`
4. Abrir trace viewer: `npx playwright show-trace [path-to-trace.zip]`

EXPLORACIÓN EN TRACE VIEWER:

- Tab **"Actions"**: Timeline de cada acción (`click`, `type`, `wait`)
- Tab **"Screenshots"**: Captura visual en cada paso

- Tab **"Network"**: Requests HTTP (API calls, assets)
- Tab **"Console"**: Logs del browser
- Tab **"Source"**: DOM snapshots punto por punto

ANÁLISIS REQUERIDO:

Documentar qué información muestra cada tab y cómo ayuda en debugging.

VALIDACIÓN: Trace viewer debe abrir mostrando timeline visual completo del

Aprende: Time-travel debugging elimina guesswork: ves exactamente

qué pasó sin logs

Puntos Clave

1. **Rich Observability (observabilidad rica):** Contexto visual reduce tiempo de debugging 80%
2. **Built-in Tools (herramientas integradas):** HTML, Trace, JSON reporters
3. **Zero Maintenance (cero mantenimiento):** Equipo de Playwright mantiene
4. **Métricas Automáticas:** Success (éxito), performance (rendimiento), compatibility (compatibilidad)
5. **No Custom Tools (sin herramientas personalizadas):** Usar lo que ya existe
6. **Fast Debugging (depuración rápida):** 3 horas → 30 minutos
7. **CI/CD Integration (integración CI/CD):** Artifact uploads automáticos